

B. E. SEMESTER END EXAMINATION (SEE)
5th SEMESTER – December 2012

10 CS 303 : COMPILER DESIGN (For ISE Only)

Time: 3 Hrs		Answer All Questions	Max Marks: 100
1.	a)	With a neat figure show the representation of the statement $a = b + c * 5$ after each phase as the translation progresses in a compiler. Note: Kindly show diagrammatically only.	8
	b)	In a string of length n , how many combinations of the following are possible? Illustrate with an example each. i) prefix ii) suffix iii) substring iv) subsequence	4
	c)	Consider the given grammar: $S \rightarrow (L) \mid a$ $L \rightarrow L, S \mid S$ i) Does left factoring make this grammar suitable for top down parsing? Suggest suitable reason for the same and how you overcome it if not. ii) Is the resulting grammar obtained after performing above step suitable for top down parsing?	4
	d)	Write a YACC specification to perform the operations of the desk calculator which is capable of handling error recovery.	4
2.	a)	Compute First (X) and Follow (A) for the given grammar applying suitable algorithms. $S \rightarrow iEtSS^l \mid a$ $S^l \rightarrow eS \mid \epsilon$ $E \rightarrow b$ Note: Algorithms may not be shown in the paper.	4
	b)	Justify is the given grammar ambiguous on input $a + b * c$. Convert the above grammar to equivalent unambiguous grammar if found ambiguous. $R \rightarrow R + R \mid R . R \mid R * \mid (R) \mid a \mid b \mid c$	4
	c)	Show that the given grammar below is a CLR grammar. Show the suitable moves of CLR parser on the input $w = aa + a^*$. $S \rightarrow SS+ \mid SS^* \mid a$ Note: Use the following value for First (S)={ a } and Follow(S) = { + , * , a , S }	12
3.	a)	Explain with an example the different parameter passing techniques.	8

b)	Define L-Attributed definitions with an example. Justify whether the given SDD is L- attributed definition or not.	6						
<table><tr><th>PRODUCTION</th><th>SEMANTIC RULES</th></tr><tr><td>$A \rightarrow LM$</td><td>$L.i := l(A.i)$ $M.i := m(L.s)$ $A.s := f(M.s)$</td></tr><tr><td>$A \rightarrow QR$</td><td>$R.i := r(A.i)$ $Q.i := q(R.s)$ $A.s := f(Q.s)$</td></tr></table>			PRODUCTION	SEMANTIC RULES	$A \rightarrow LM$	$L.i := l(A.i)$ $M.i := m(L.s)$ $A.s := f(M.s)$	$A \rightarrow QR$	$R.i := r(A.i)$ $Q.i := q(R.s)$ $A.s := f(Q.s)$
PRODUCTION	SEMANTIC RULES							
$A \rightarrow LM$	$L.i := l(A.i)$ $M.i := m(L.s)$ $A.s := f(M.s)$							
$A \rightarrow QR$	$R.i := r(A.i)$ $Q.i := q(R.s)$ $A.s := f(Q.s)$							
c)	Describe the various dynamic allocation strategies with a neat example.	6						
a)	Draw an annotated parse tree and a dependency graph for the input $(3+4)*(5+6)n$ using the grammar below: $L \rightarrow En$ $E \rightarrow E + T \mid T$ $T \rightarrow T * F \mid F$ $F \rightarrow (E) \mid \text{digit}$	6						
b)	A Boolean expression can be translated into intermediate form without generating code for any of the Boolean operators and as well without having the code necessarily to evaluate entire expression. Achieve the same by representing the value of the given Boolean expression below by their position in code sequence. If $a < b$ or $c < d$ and $e < f$	6						
c)	Calculate the instruction cost for the following statements: i) Mov R0, R1 ii) LD R0, a(R1) iii) Sub 5(R0) , *10(R1) iv) ADD R0, R0, R1 ST X1, R0	4						
d)	Define a viable prefix. Find the viable prefixes for the sentential form $E+P^{\wedge}(a+a)\#$ for the given grammar: $S \rightarrow E\#$ $E \rightarrow T$ $E \rightarrow E + T$ $T \rightarrow P$ $T \rightarrow P^{\wedge} T$ $P \rightarrow F$ $P \rightarrow P * F$ $F \rightarrow a$ $F \rightarrow (E)$	4						

a) Consider the program fragment below.

```
p=0;  
i=1 do  
    p=p + A[i] * B[i]  
    i=i+1  
while(i<=20)
```

Perform the following operations on the code mentioned above:

- i) Generate intermediate code for the sequence.
- ii) Construct basic blocks and flow graphs for the same.
- iii) Apply code motion and reduction in strength and elimination of induction variables Optimization techniques for the same.

16

b) Construct a DAG for the expression given below:

$((x + y) - ((x + y) * (x - y)) + (x - y) * (x - y))$

4